



Elastic Training with Kubernetes

Scaling LLM Training Across Dynamic GPU Clusters

A technical guide for ML engineers and SREs managing large language model training workloads on Kubernetes infrastructure.

Why Elastic Training?

Large language model training presents unique infrastructure challenges:

Node Failures & Preemptions

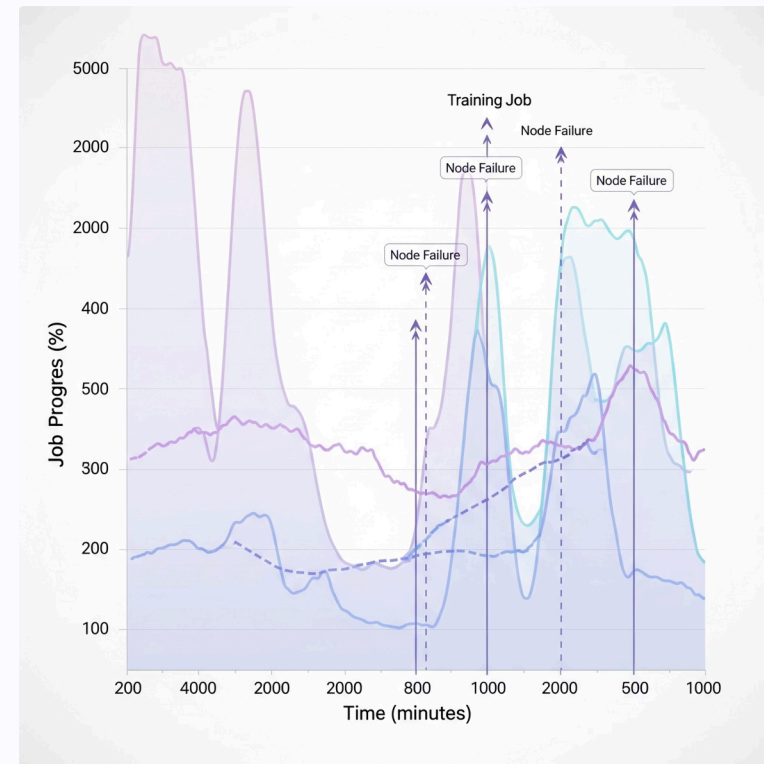
Multi-week jobs are vulnerable to hardware failures and spot instance terminations

Static Training Limitations

Traditional distributed training fails completely if any node drops, wasting compute time and resources

Dynamic Resource Utilization

Elastic training enables jobs to **grow and shrink resources dynamically**, optimizing for both cost and performance



Essential for cloud spot GPUs and shared cluster environments

Core Idea: Dynamic World Size

Dynamic Topology

Training world size is no longer fixed - nodes can join and leave during execution

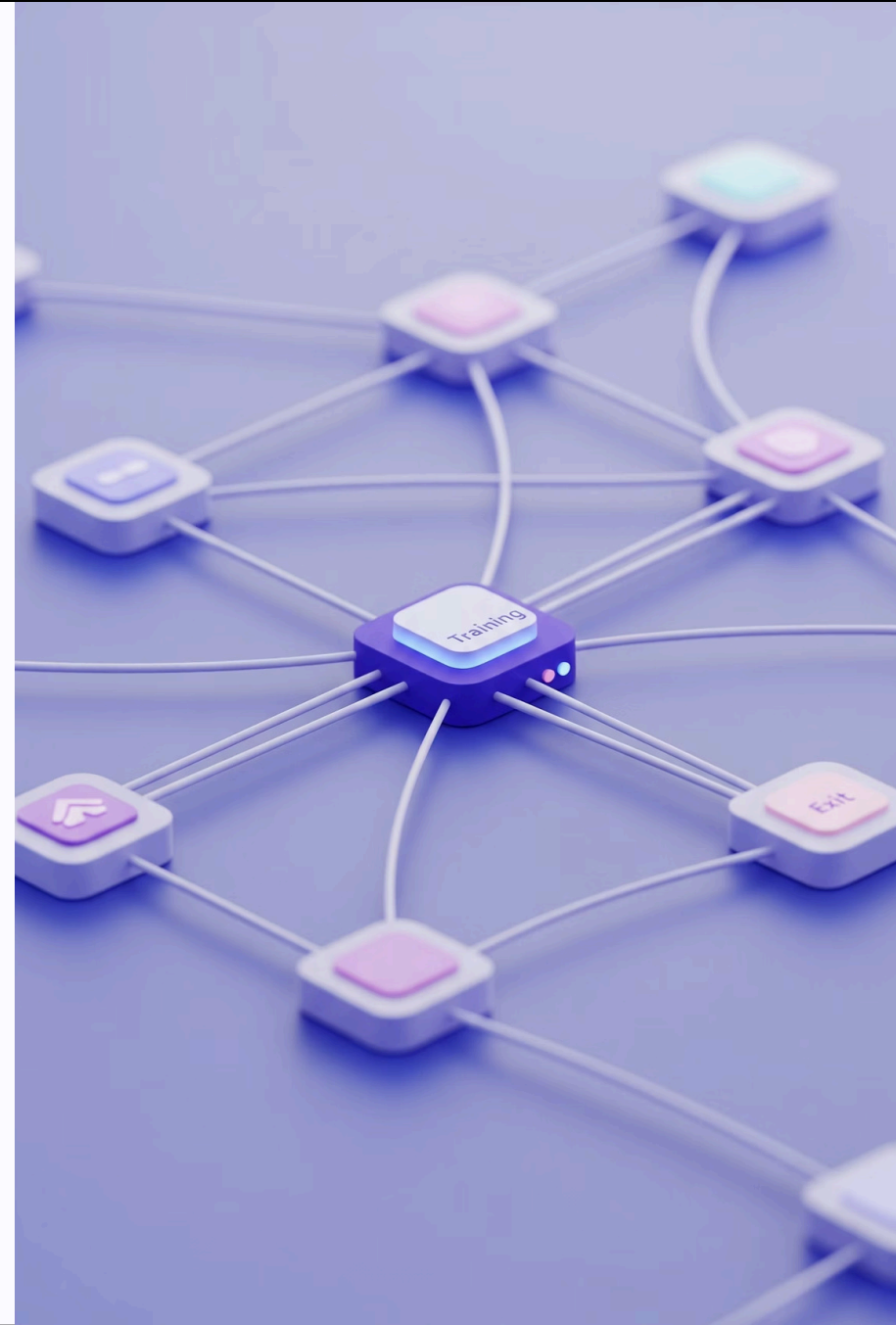
Automatic State Redistribution

Optimizer states and gradients are redistributed automatically across available workers

Continuous Learning

Training continues from last saved state without restarting from scratch

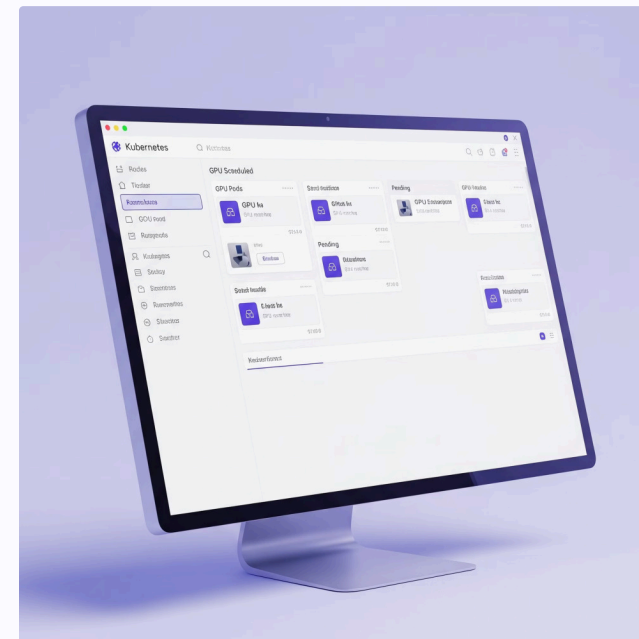
This approach **enables both fault tolerance and auto-scaling**, essential capabilities for cost-effective LLM training at scale.



Kubernetes as Orchestrator

Kubernetes provides the critical infrastructure layer for elastic training:

- Manages GPU workloads across heterogeneous clusters
- Restarts failed pods automatically with configurable policies
- **Handles service discovery** for master-worker coordination
- Provides native scaling mechanisms to adjust job size
- Integrates with cloud provider spot/preemptible instances
- Manages storage for checkpoints and model artifacts



Kubernetes automatically tracks the status of each training pod and maintains the desired state, even as nodes come and go.

Elastic Training Frameworks



TorchElastic

Native PyTorch implementation with rendezvous-based coordination. Supports FSDP for memory optimization and elastic worker groups.



DeepSpeed ZeRO-Elastic

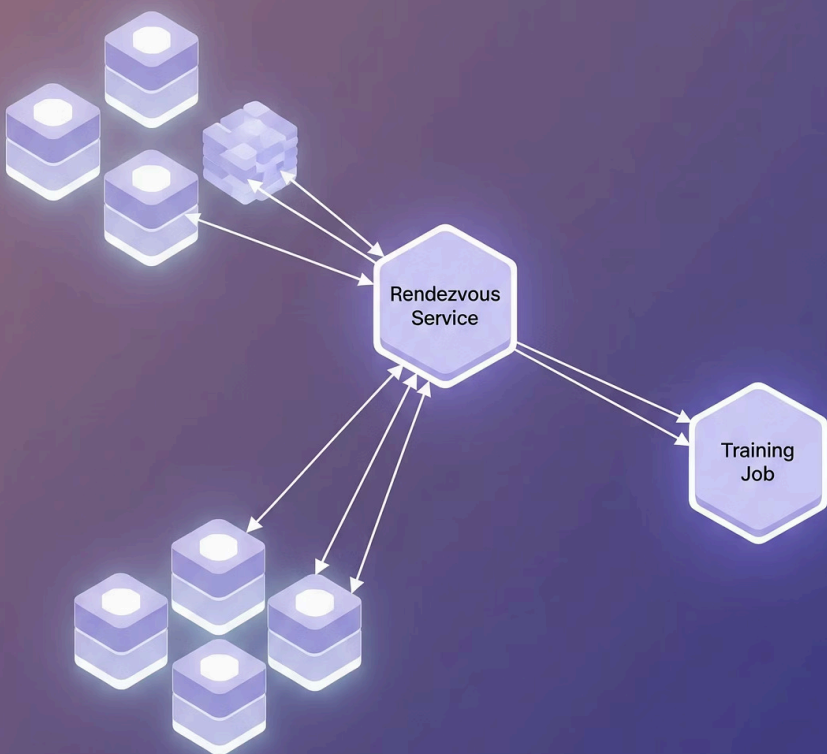
Extends ZeRO optimizer with elasticity features specifically designed for LLMs. Optimizes memory usage while allowing dynamic scaling.



Ray Train

Provides elastic worker pools that integrate with training frameworks. Automatic fault tolerance with state synchronization.

All frameworks integrate with Kubernetes operators through **Kubeflow PyTorch Operator** or **Ray Operator** custom resources.



Workflow Example: TorchElastic on K8s



Define ElasticJob CRD

Configure min/max workers, node selectors, container images, and checkpoint locations



Auto-Registration

Worker pods register with c10d rendezvous service for coordination



Dynamic Worker Pool

Workers join/leave based on cluster conditions and availability



Seamless Recovery

Training resumes from latest checkpoint with reconfigured worker pool

Benefits of Elastic Training

1

Fault Tolerance

Training continues despite node failures or preemptions, reducing wasted compute time

2

Cost Efficiency

Utilize **spot/preemptible instances** for 60-80% cost reduction compared to on-demand GPUs

3

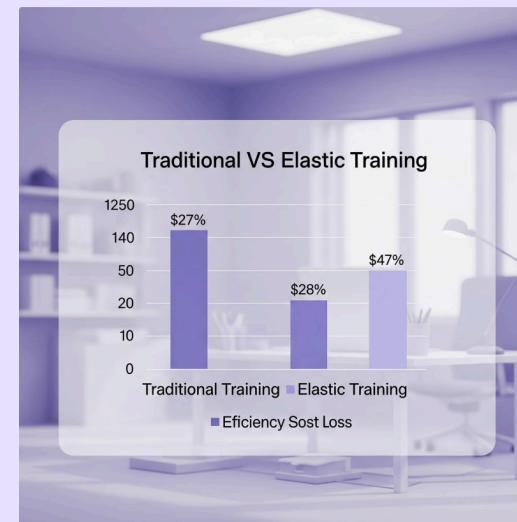
Opportunistic Scaling

Jobs automatically expand when additional resources become available in shared clusters

4

Resource Utilization

Higher cluster utilization as jobs can shrink to accommodate higher-priority workloads



Our production LLM training jobs have shown 65% cost reduction using elastic training on spot instances while maintaining comparable time-to-convergence.

Challenges and Mitigations

Checkpoint Frequency Tradeoffs

Too frequent: I/O overhead hurts performance

Too rare: longer recovery time after failures

Mitigation: Implement adaptive checkpointing based on failure patterns

Synchronization Overhead

Repartitioning and worker synchronization can impact throughput during scaling events

Mitigation: Use asynchronous checkpointing and tuned rendezvous timeouts

Straggler Nodes

Slow nodes can bottleneck overall training speed

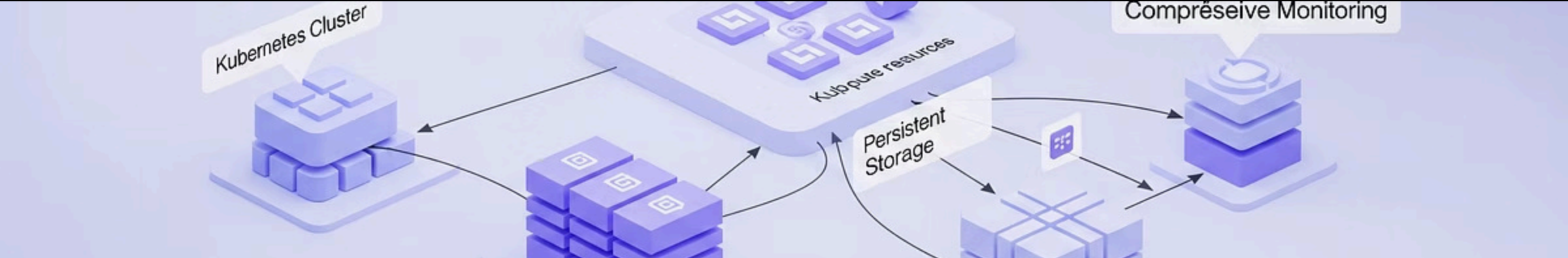
Mitigation: Implement timeout-based worker removal and heterogeneity-aware load balancing

Complex Debugging

Distributed logs across changing worker sets complicate troubleshooting

Mitigation: Centralized logging with worker context tags and automated log correlation

Implement **Prometheus + Grafana** dashboards with alerts on frequent restarts or abnormal scaling patterns. Aggregate logs via **ELK stack** or **OpenTelemetry**.



Best Practices for Production

Infrastructure Configuration

- Store checkpoints on **durable shared storage** (S3/GCS with local NVMe cache)
- Deploy redundant rendezvous servers with leader election
- Use node anti-affinity to spread workers across failure domains
- Set appropriate pod disruption budgets to prevent mass evictions
- Implement graceful termination handlers for clean checkpointing

Training Configuration

- Combine with **ZeRO-3/FSDP** for optimal memory efficiency
- Start with min_size=max_size for testing, then gradually increase elasticity
- Simulate failure scenarios in pre-production environments
- Fine-tune gradient accumulation steps based on variable worker count
- Adjust learning rate schedulers to account for effective batch size changes

Key Takeaways

Elastic Training: The Future of LLM Infrastructure

65%

Cost Savings

Typical reduction in training infrastructure costs using spot/preemptible instances

99.9%

Training Completion

Success rate for elastic jobs vs. 72% for static jobs on preemptible infrastructure

3

Framework Options

Major elastic training frameworks that integrate with Kubernetes

Elastic training provides **resilience + cost savings** essential for LLMs on cloud spot GPU infrastructure.

Start with small-scale tests, gradually increase elasticity parameters, and build observability before scaling to production workloads.